

Can Multi-LLM Negotiation Improve Scheduling Decisions? A Policy-Selector Referee for Malleable GPU Clusters

Lay Kim Seng

Cyberscience Center, Tohoku University
Sendai, Miyagi, Japan
lay.kim.seng.q4@dc.tohoku.ac.jp

Hiroiyuki Takizawa

Cyberscience Center, Tohoku University
Sendai, Miyagi, Japan
takizawa@tohoku.ac.jp

Abstract—GPU scheduling requires decisions based on interacting information such as resource availability, job priority, waiting time, deadline risk, and expected execution progress. Large Language Models (LLMs) offer a flexible mechanism for reasoning over such heterogeneous scheduling information, but directly generating resource allocations with an LLM does not provide deterministic feasibility guarantees. This paper proposes a multi-LLM *Policy-Selector Referee* architecture for malleable GPU scheduling. A Demand Agent evaluates scheduling conditions from the perspective of individual jobs, while a Supply Agent evaluates cluster-wide resource efficiency and scarcity. A Referee reconciles both views and selects a deterministic scheduling policy rather than directly calculating GPU allocations. The selected global policy determines how jobs compete for resources, while per-job sizing decisions determine whether malleable jobs retain, reduce, or increase their GPU allocations. Conventional scheduler code performs the final allocation and enforces resource constraints. The framework is evaluated using trace-driven workloads derived from the MIT Supercloud dataset and simulated with ElastiSim. Counterfactual simulation is used to generate oracle policy labels for parameter-efficient fine-tuning. The evaluation separately tests the effects of market-based scheduling mechanisms and explicit Demand–Supply role separation.

Index Terms—GPU scheduling, large language models, multi-agent reasoning, market-based scheduling, policy selection, malleable workloads, high-performance computing

I. INTRODUCTION

GPU-accelerated systems execute heterogeneous scientific and machine-learning workloads with different resource demands, execution times, and priorities [7]. Because GPU capacity is finite, a scheduler must determine which jobs should run, when they should start, and how available resources should be distributed among competing jobs.

These decisions involve competing objectives. From the *demand side*, individual jobs benefit from shorter waiting times, faster execution, and lower service-level risk. From the *supply side*, the cluster must use limited resources efficiently while avoiding unnecessary idle capacity and starvation. Traditional HPC schedulers address such trade-offs using deterministic mechanisms such as FCFS and backfilling [1].

The scheduling problem becomes more flexible, but also more complex, when jobs are *malleable*. Unlike rigid jobs,

malleable jobs may change their resource allocation during execution [6]. A scheduler must therefore decide not only which jobs should receive priority, but also whether a running job should expand, shrink, or retain its current allocation.

Learning-based methods have previously been applied to cluster scheduling. DeepRM formulates resource management as a deep reinforcement-learning problem [2], while Decima learns scheduling and parallelism decisions for data-processing workloads [3]. More recently, LLMs have also been investigated for scheduling. LarS, for example, adapts an LLM using scheduling trajectories for cost-aware task scheduling [5].

This line of work suggests that LLM decisions can be conditioned on multiple forms of scheduling information provided together in the same context [5]. Such flexibility is attractive when the importance of different scheduling factors changes with system state. However, an LLM output does not itself provide a mathematical guarantee that hard resource constraints are satisfied. We therefore separate flexible reasoning from numerical allocation.

A second question concerns how that reasoning should be structured. Multi-agent debate has shown that assigning distinct reasoning roles to language-model instances can improve reasoning in other domains [8]. Inspired by this idea, we explicitly separate job-oriented and system-oriented objectives into Demand and Supply roles.

Accordingly, this study investigates whether explicit Demand–Supply reasoning, reconciled by an LLM Referee that selects deterministic scheduling policies, improves malleable GPU scheduling decisions relative to a single LLM receiving the same scheduling information. We also investigate whether market-based scheduling mechanisms provide additional benefits under resource contention and whether those mechanisms interact with Demand–Supply role separation.

II. RELATED WORK

A. Traditional and Learning-Based Scheduling

Traditional HPC schedulers rely on deterministic rules such as FCFS, priority scheduling, and backfilling. EASY backfilling permits smaller jobs to execute ahead of the queue head provided that doing so does not delay the reservation of the

head job [1]. Such mechanisms provide predictable resource-management behavior because their decisions follow explicitly defined rules.

Malleable scheduling extends conventional batch scheduling by allowing the allocation of a running application to change during execution. ElastiSim provides a batch-system simulation environment designed specifically for studying malleable workloads and scheduling strategies [6].

Learning-based scheduling provides another approach to adaptive decision-making. DeepRM learns resource-management decisions using deep reinforcement learning [2], while Decima learns scheduling decisions for data-processing clusters [3]. Because learned policies can be difficult to interpret directly, later work has also investigated extracting interpretable scheduling rules from learned models [4].

B. LLM-Assisted Scheduling

Recent work has explored LLMs as scheduling decision makers. LarS uses high-quality scheduling trajectories to adapt an LLM for cost-aware task scheduling [5]. This demonstrates that scheduling state can be represented as model input and mapped to scheduling decisions.

Our approach changes the role of the LLM. Rather than asking the model to generate an executable resource assignment, the LLM selects from a constrained set of deterministic scheduling policies. Conventional code then performs allocation arithmetic and hard constraint enforcement.

C. Multi-Agent Reasoning

Multi-agent debate allows language-model instances with different roles to examine competing arguments before a final decision is produced. Du *et al.* showed that this interaction can improve reasoning performance in language-model tasks [8].

We adapt this principle to GPU scheduling through three roles. The Demand Agent represents job-level objectives, the Supply Agent represents cluster-level objectives, and the Referee determines which deterministic scheduling behavior should govern the current state. The benefit of this role separation is treated as an experimental hypothesis rather than assumed *a priori*.

III. SYSTEM ARCHITECTURE

The proposed architecture separates strategic reasoning from numerical resource allocation. At each scheduling epoch, the system observes the current GPU capacity, pending jobs, running malleable jobs, waiting times, priorities, estimated remaining work, and other available scheduler state.

The decision process contains four stages. First, the current workload and resource state are collected. Second, Demand and Supply Agents independently evaluate the state. Third, the Referee selects a global scheduling policy and per-job sizing behavior. Finally, deterministic scheduler code translates those decisions into a feasible GPU allocation.

A. Demand and Supply Reasoning

The **Demand Agent** evaluates the state from the perspective of individual jobs, considering information such as waiting time, priority, deadline pressure, and expected benefit from additional GPUs.

The **Supply Agent** evaluates the same state from the perspective of the cluster, considering GPU scarcity, queue pressure, fairness, and the efficiency of reallocating resources.

Both agents receive the same observable scheduling state and return structured assessments. Neither agent directly calculates GPU assignments. The Referee receives both assessments together with the original scheduler state.

B. Global Policy and Per-Job Sizing

The Referee makes two related decisions.

First, it selects one *global scheduling policy* for the current decision epoch. This policy determines how competing jobs are prioritized when GPU resources are scarce.

Second, it selects a sizing or resizing behavior for each malleable job. A job may retain its requested allocation, adapt its allocation to share resources, expand when spare GPUs are available, or avoid resizing when the expected benefit is insufficient.

The global policy space is divided into two families.

The **non-market** family contains conventional deterministic priority mechanisms:

- **LLF**: prioritizes jobs with greater deadline risk;
- **FAIR**: prioritizes jobs or users receiving relatively less service; and
- **PROD**: prioritizes configured high-priority workloads.

The **market-based** family resolves contention using explicit job value or incentives:

- **AUCTION**: allocates scarce resources through deterministic bidding and clearing;
- **BID-DENSITY**: prioritizes jobs according to declared or derived value per unit of GPU demand; and
- **INCENTIVE**: adjusts allocation priority using an explicit credit or incentive mechanism.

FCFS is retained as a fixed deterministic baseline rather than included in either adaptive policy family.

The same per-job sizing/resizing action space is used with both policy families so that market and non-market mechanisms can be compared without changing the treatment of malleable jobs.

C. Gated Invocation and Safety

Routine scheduling states may be handled by a deterministic default policy without invoking an LLM. The reasoning path is activated when configured conditions indicate that additional policy selection may be useful, for example under high queue pressure, deadline risk, conflicting Demand and Supply signals, or contextual information not represented by the default rule.

After policy selection, deterministic scheduler code computes the resource assignment. Every malleable job must remain within its configured resource bounds,

$$g_i^{\min} \leq g_i \leq g_i^{\max}, \quad (1)$$

and the total allocation must remain within cluster capacity. Invalid or malformed Referee outputs are rejected and replaced by the deterministic fallback policy.

IV. METHODOLOGY

A. Trace-Driven Malleable Workload

The workload is derived from the MIT Supercloud Dataset, which contains Slurm accounting records collected from a production HPC environment [7]. The workload is replayed with ElastiSim, a batch-system simulator supporting malleable workloads [6].

Scheduling is modeled at GPU granularity, with one schedulable ElastiSim resource unit representing one GPU. This is a resource abstraction and does not imply that each simulated unit corresponds to a complete physical server.

The trace records an observed GPU allocation and runtime for each job but does not provide the runtime of the same execution under every alternative GPU allocation [7]. Counterfactual performance is therefore modeled using a generalized Amdahl-style speedup function,

$$S_i(g) = \frac{1}{(1 - p_i) + p_i/g^{s_i}}, \quad (2)$$

where p_i represents the parallel fraction and s_i controls sublinear scaling. The formulation is based on the classical relation between parallel fraction and speedup introduced by Amdahl [9].

Each simulated job is anchored to its observed GPU allocation and runtime so that the historical runtime is reproduced at the original operating point. Only performance at unobserved GPU allocations is introduced by the counterfactual scaling model. Multiple scaling regimes are evaluated to reduce dependence on one assumed speedup curve.

B. Synthetic Deadlines

The workload representation used in this study does not provide an explicit user deadline for every job. Synthetic deadlines are therefore introduced as experimental service-level targets rather than treated as observed trace information:

$$D_i^{\text{synth}} = a_i + \alpha_i W_i^{\text{req}}, \quad (3)$$

where a_i is arrival time, W_i^{req} is requested walltime, and α_i controls deadline slack. Deadline parameters are fixed before scheduler evaluation and tested across multiple slack regimes.

C. Oracle Policy Labels and Fine-Tuning

The Referee is trained using policy labels generated through counterfactual simulation rather than manual annotation. For each sampled scheduling state, every candidate policy in the relevant policy family is evaluated from the same initial condition over a fixed rollout horizon. The best-performing policy becomes the oracle label:

TABLE I
MAIN FACTORIAL EVALUATION

	Non-Market	Market-Based
Single Referee	Single-NM	Single-MKT
Demand-Supply + Referee	Multi-NM	Multi-MKT

$$P_t^* = \arg \max_{P_k \in \mathcal{P}} R(S_t, P_k). \quad (4)$$

The evaluation score considers service-level violations, bounded slowdown, queueing delay, resizing overhead, and useful GPU progress. Reward parameters are fixed using validation data.

Each training example contains the observable scheduler state and its oracle policy label. In the multi-agent condition, the Demand and Supply assessments are additionally provided to the Referee. Low-Rank Adaptation (LoRA) is used for parameter-efficient fine-tuning [10]; LoRA-based adaptation has also been applied to LLM scheduling in prior work [5].

Scheduling states are collected from multiple workload windows and partitioned at the *window level* into training, validation, and test sets. States from the same workload window are never distributed across multiple partitions.

V. EXPERIMENTAL DESIGN

The evaluation separates the effect of the scheduling mechanism from the effect of Demand-Supply role separation. This distinction is necessary because superior performance of a market-based policy alone would not demonstrate that multi-agent reasoning is beneficial.

A. Research Hypotheses

The experiments evaluate three hypotheses:

- **H1—Market effect:** market-based policy selection improves scheduling performance relative to non-market policy selection under resource contention;
- **H2—Role-separation effect:** Demand-Supply reasoning improves policy selection relative to a single Referee receiving the same scheduler state; and
- **H3—Interaction effect:** Demand-Supply separation provides a larger benefit when selecting among market-based policies than when selecting among non-market policies.

B. Main Factorial Experiment

The main evaluation follows the 2×2 design shown in Table I.

Single-NM and **Single-MKT** use one Referee that receives the complete scheduler state and selects from the corresponding policy family.

Multi-NM and **Multi-MKT** provide the same state to separate Demand and Supply Agents before Referee selection.

All four conditions use the same underlying LLM, workload states, per-job sizing actions, deterministic execution layer, and feasibility validator. The only experimental factors are the available global policy family and the reasoning structure.

Comparing Single-MKT with Single-NM, and Multi-MKT with Multi-NM, tests the effect of market-based mechanisms. Comparing Multi-NM with Single-NM, and Multi-MKT with Single-MKT, tests the effect of Demand–Supply separation while holding the policy family constant.

If the improvement from Single-MKT to Multi-MKT is larger than the improvement from Single-NM to Multi-NM, the result provides evidence that Demand–Supply reasoning interacts particularly strongly with market-based policy selection.

C. Baselines and Secondary Comparisons

The four primary conditions are compared with:

- FCFS as a conventional fixed baseline;
- the best fixed non-market policy selected on validation data;
- the best fixed market policy selected on validation data;
- Decision Tree and Logistic Regression policy selectors;
- zero-shot LLM policy selection;
- oracle selectors for the market and non-market policy families; and
- a unified Referee allowed to select from both policy families.

The unified selector is treated as a secondary experiment. It tests whether the Referee can learn when a market mechanism should be used without requiring a manually designed threshold between market and non-market operation.

D. Evaluation Metrics

All methods are evaluated on identical held-out workload windows and paired simulation seeds. The main scheduling metrics are:

- synthetic deadline violation rate;
- bounded slowdown;
- mean waiting time;
- useful GPU utilization;
- number of completed jobs; and
- resizing overhead.

Policy-selection quality is additionally measured by the performance gap between the Referee-selected policy and the counterfactual oracle policy.

Paired statistical comparisons are performed across workload windows. When multiple pairwise hypotheses are tested simultaneously, corrected p -values are reported.

VI. CONCLUSION

This paper proposes a Policy-Selector Referee architecture for malleable GPU scheduling. Instead of directly generating resource allocations, LLMs reason over the scheduler state and select among predefined deterministic policies, while conventional code remains responsible for allocation arithmetic and hard feasibility constraints.

The architecture separates two levels of decision-making. A global policy determines how jobs compete for scarce resources, while per-job sizing and resizing decisions determine how malleable jobs use their assigned capacity. Demand

and Supply Agents provide explicit job-level and cluster-level perspectives to the Referee.

The experimental design separately evaluates market-based scheduling and Demand–Supply role separation. This allows an improvement caused by an auction or incentive mechanism to be distinguished from an improvement caused by multi-agent reasoning. Counterfactual simulation additionally provides oracle policy labels for fine-tuning and an upper bound for the candidate policy space.

REFERENCES

- [1] D. A. Lifka, “The ANL/IBM SP scheduling system,” in *Job Scheduling Strategies for Parallel Processing*, 1995, pp. 295–303.
- [2] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. ACM HotNets-XV*, 2016.
- [3] H. Mao *et al.*, “Learning scheduling algorithms for data processing clusters,” in *Proc. ACM SIGCOMM*, 2019, pp. 270–288.
- [4] Z. Hu, C. Wang, H. Paik, Y. Shu, and L. Zhu, “Learning interpretable scheduling algorithms for data processing clusters,” *arXiv preprint arXiv:2405.19131*, 2024.
- [5] H. Pei *et al.*, “LLM-based cost-aware task scheduling for cloud computing systems,” *J. Cloud Comput.*, vol. 14, Art. no. 81, 2025.
- [6] T. Özden, T. Beringer, A. Mazaheri, H. M. Fard, and F. Wolf, “ElastiSim: A batch-system simulator for malleable workloads,” in *Proc. 51st Int. Conf. Parallel Processing (ICPP)*, 2022, pp. 1–11.
- [7] S. Samsi *et al.*, “The MIT Supercloud Dataset,” in *Proc. IEEE High Performance Extreme Computing Conf. (HPEC)*, 2021, pp. 1–8.
- [8] Y. Du *et al.*, “Improving factuality and reasoning in language models through multiagent debate,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2024.
- [9] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [10] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.